StudentShield: Manchester.Ai_IBM Z Datathon 2026 Project Report

Oct 18, 2026 · Thinojen Balakumaresan, Julian Newson, Sshubhan Kammari, Arda Yencer

Summary

StudentShield catches money-mule payments the moment they land and warns the account holder before they send the money on to another account.

Built by team **Manchester.Ai** for the theme **Real-time AI for critical decisions**, it runs entirely on IBM LinuxONE (s390x). An XGBoost model trained on IBM's own anti-money-laundering dataset scores each incoming payment in about a millisecond. A two-level alert system then decides who is told: the bank's analyst, or the student as well. Each alert carries plain-English reasons taken from the model's own feature contributions, which become a student warning and an analyst case note: written from templates built from those same reasons. The service can pass the reasons to IBM Granite instead, but no Granite server ran in the final build.

Headline result (test days 9–10 Sep, unseen)	Value
Laundering caught when flagging the riskiest 1% of payments	**69.5%**, vs 25.7% for a hand-written rules baseline (2.7×)
PR-AUC	**0.430**, vs 0.024 for the rules (18×)
Within ACH payments only (rules collapse to 2.0%)	**53.6%** recall at 42.0% precision
Student warnings that are real mule payments	**69.7%** (7 in 10), covering 37.8% of all laundering
Analyst alerts that are real	**49.5%** (1 in 2), covering 45.2% of all laundering
Training time on LinuxONE	XGBoost **19.0 s** on 2.09M payments
Scoring latency on LinuxONE	about **1 ms** per payment

- **Repository:** ksshubhan/manchester-imbz-datathon-2026
- **Event code:** SSF2026

Judging criteria

Where to find the evidence for each judging criterion (10 points each, 40 in total).

Criterion	Where the evidence is
1. Value to the community and theme alignment	Theme 1, real-time AI: scoring at about 1 ms as each payment lands (Summary). A sourced problem in the age group most affected (The problem). Clear beneficiaries: students warned before they commit a crime, victims whose money can still be held, analysts with smaller, better queues (Our solution, Alert levels)
2. Novelty and originality	Warn the student first, in the window before money moves on. Two alert levels with cut-offs chosen by who bears the cost of a mistake (F2 and F0.5). Phone rules for re-asking and stopping. A demo of a real case, not a script. All code written during the event: 80 commits from 11:26 on 17 Oct (Decisions, Roads not taken)
3. Technical design and use of LinuxONE	Training and inference on s390x. Three containers

Criterion	Where the evidence is
	with the model isolated on a private network. XGBoost on 2.09M payments in 19.0 s with no GPU. Point-in-time features, a time split, a test set used once (Architecture, Running it on LinuxONE including Containerisation, security and scalability, Evaluation method). Labs: predictive multi-model (XGBoost + Isolation Forest, compared and blended) and predictive + generative (both IBM labs completed; in our build the model's reasons become plain English through templates, with an IBM Granite hook ready but not running)
4. Execution, delivery and pitch	A working web app with a landing page, a step-by-step real case and a full replay at scale. A clear pitch line: a smoke alarm, not a fire investigation
AI Innovation Excellence Awards (extra, originality + impact and technical excellence)	Explanations that can't invent a reason (built only from SHAP reasons). The "mules hide in plain sight" Isolation Forest finding. The ACH-only test. Honest drift reporting

The problem

Young people are heavily overrepresented among money mules, and banks usually catch a mule only after the money has gone.

A money mule is the middle link in a laundering chain. Criminal proceeds land in their account and they are told to send the money on quickly, often within hours, sometimes for a cut. Students are recruited with "easy money" offers, and many don't realise it's a crime: Cifas research found that 22% of UK adults thought money muling was legal (Financial IT).

Figure	What it shows	Source
23%	of suspected mule cases on the Cifas database involved young people aged 21 and under (H1 2023)	UK Finance and Cifas; also STV News
64%	of those cases involved people aged up to 30 (H1 2023)	UK Finance and Cifas
37,000	UK bank accounts showed mule-like behaviour in 2023 (Cifas estimate)	Financial IT (Jan 2025)
£10bn+	laundered through mule activity in the UK each year (NCA estimate)	Financial IT (Jan 2025)
£1.28bn	lost to payment fraud in the UK in 2025, up 4%	UK Finance Annual Fraud Report 2026, via FinTech Futures
£576.4m	lost to authorised push payment (APP) fraud in 2025, up 19%; mule accounts are where this money lands	UK Finance Annual Fraud Report 2026, via FinTech Futures

Money laundering carries up to 14 years in prison. When a bank finds a mule today, the criminal chain is usually complete: the victim's money is gone, and the student faces a closed account, a fraud marker that can block future banking, or prosecution. Nothing reached them in the hours when they could still have stopped.

Our own evidence: we planned a short survey of students on “easy money” offers but did not have time to run it in 24 hours; it is the first thing we would add.

Our solution

StudentShield acts in the window between money arriving and money leaving. It scores every incoming payment, and only interrupts the student when the model is confident.

Think of a smoke alarm rather than a fire investigation: the alarm goes off while there is still time to get out.

	Usual approach	StudentShield
When the mule is caught	After the money has moved from middleman to recipient	When the money lands, before it is sent on
Who hears first	Bank fraud team	The analyst, and the student once the model is confident
Outcome for the student	Account closed, possible fraud marker	A calm question and a route to help
Outcome for the chain	Already complete	Broken while funds can still be held
What the analyst gets	A risk score	A score, its rarity (“1 in N”) and three plain-English reasons

Two alert levels, because two people bear different costs. An analyst can absorb false alarms; a frightened innocent student cannot. So the same model score triggers two cut-offs:

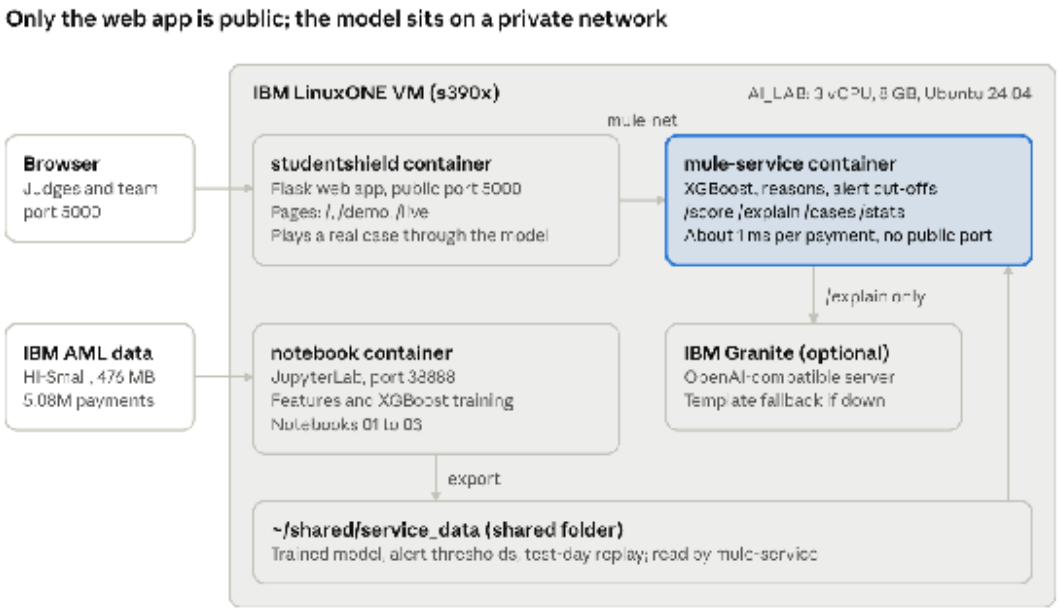
- **Analyst alert** (recall-weighted, F2): the payment appears on the bank’s console with reasons. The student sees nothing.
- **Student warning** (precision-weighted, F0.5): the account holder is also asked, “Has someone asked you to move this money?”, and transfers out of the account can be held.

Phone rules. The team agreed these at 16:43 on 17 October, and both demo pages follow them:

1. Below the analyst cut-off, the phone stays normal.
2. Analyst level only: the phone still stays normal. About half of these alerts are innocent (49.5% precision), too many to contact students.
3. Student level: the warning screen offers three answers: *Pause, I need help*; *Someone asked me to move it*; *This is legitimate*. The question is about all the money in the account, so one answer covers every payment.
4. After an answer, any new **flagged** payment into the account asks “Does your answer still stand?”. Unflagged payments don’t re-ask.
5. Transfers out are held only once the account has reached the student level. The student cannot refuse incoming money, but the bank can hold money going out.
6. The analyst view shows every analyst- and student-level alert, plus every answer the student gave. The phone never shows scores, reasons or the dataset’s answer key.
7. Asking stops after *Someone asked me to move it*, or after 3 answers. The phone then shows “A specialist is handling this”.

Architecture

Three containers on one LinuxONE VM: Jupyter trains the model, mule-service scores payments on a private network, and studentshield is the only part exposed to the internet.



StudentShield on IBM LinuxONE · 3 containers, 1 VM

StudentShield on IBM LinuxONE · 3 containers, 1 VM

The notebook exports the trained model, thresholds and test-day replay to a shared folder that mule-service reads at start. The web app sends each payment to mule-service over mule-net. Granite is shown as optional: it would be called only when an explanation is opened, and no Granite server ran in the final build, so explanations come from templates.

Data

We use IBM's synthetic anti-money-laundering dataset (HI-Small): 5.08 million payments, of which 1 in 1,000 is laundering. We keep the 1–10 September window, where normal and laundering activity coexist.

Dataset. IBM Transactions for Anti-Money Laundering, HI-Small (HI-Small_Trans.csv, 476 MB), loaded on IBM LinuxONE in about 20 s (1.42 GB in memory). Notebook: notebooks/01_cleaning.ipynb.

Fact	Value
Payments	5,078,345
Accounts	515,088
Laundering payments	5,177 (0.10%), about 1 in 1,000
Accounts that received laundered money	3,984 (our mule candidates)
Date range	1 Sep 2022 to 18 Sep 2022
Self-transfers (same sender and receiver)	591,212 (11.6%), only 11 of them laundering

Laundering by payment format. ACH bank-to-bank transfers are 12% of payments but carry 87% of the laundering.

Format	Payments	Laundering	Laundering rate
ACH	600,797	4,483	0.75%
Cheque	1,864,331	324	0.02%
Credit card	1,323,324	206	0.02%
Cash	490,891	108	0.02%
Bitcoin	146,091	56	0.04%
Reinvestment	481,056	0	0%
Wire	171,855	0	0%

By currency, the Saudi riyal stands out at 0.42% laundering, about 4× the average.

Cleaning decisions.

- **Only 1–10 September is used.** From 11 September the generator stops producing normal payments while laundering chains carry on (on 18 September, 8 of the 11 payments are laundering). Testing on those days would make any model look excellent, and a judge who knows the dataset would spot it.

- **Self-transfers are dropped.** A mule receives money from someone else, and moving money between your own pots is not that. This removes 591k rows of near-pure noise and leaves about 4.5M payments.

- **The data is synthetic, and we say so.** The results show the approach works. Real-world accuracy would need real bank data.

Why this dataset

We chose IBM's AML data because it is the only large public option that models laundering itself (money placed, layered and moved on) with complete labels, in the payment formats a UK student's account sees. Within it, HI-Small fits the VM and gives the most laundering examples per gigabyte.

Why synthetic, not real bank data. Real transaction data isn't available to students, for legal, privacy and competitive reasons. Even inside a bank its labels are incomplete: undetected laundering sits in the data marked as normal, and each bank sees only its own side of a chain that crosses banks. IBM's generator knows where every pound came from, so every laundering payment is labelled, including the ones a bank would miss (Altman et al., NeurIPS 2023).

Why IBM's AML set over other public datasets.

Option	What it is	Why it fits our question worse
IBM Transactions for AML (chosen)	Synthetic bank payments across ACH, cheque, card, cash, wire, Bitcoin and reinvestment; every laundering payment labelled; patterns include fan-in, fan-out, gather-scatter and cycles	—
AMLSim	An earlier synthetic AML generator; about 1.32M transactions, transfers only	Transfers only, so no payment-format signal, and smaller (as compared in the IBM paper)
PaySim	Synthetic mobile-money transactions labelled for fraud	Labels fraud such as account takeover, not laundering, so it can't teach the mule pattern
Elliptic	Bitcoin transaction graph labelled illicit or licit	Crypto only, and its features are anonymised, so a reason can't be explained in plain English

It's also IBM's own data, running on IBM's own platform. Judges from IBM can check our numbers against a dataset they know, which is also why we were careful to drop the synthetic tail after 10 September.

Why HI (higher illicit) rather than LI (lower illicit). IBM publishes each size in two versions. LI has a lower laundering rate: the LI-Large set has about 1 laundering payment in 1,750, against about 1 in 1,000 in our HI-Small. With one day of build time, HI gives the model more of the rare cases to learn from. Real banks would see rates nearer LI, so precision would fall, and we flag this in Limitations.

Why Small rather than Medium or Large. HI-Small is 476 MB and 5.08M payments. It loaded in about 20 s and used 1.42 GB of the VM's 8 GB, leaving room for feature building and two Jupyter users at once. The Large sets hold about 175–180M transactions, roughly 35× more, which wouldn't fit in 8 GB of memory. SCOPE.md planned to "start with a sample of HI-Small, scale up only if comfortable". The full HI-Small fitted, so no sampling was needed, and HI-Medium and HI-Large are parked in later.md.

Extra files we kept for later. HI-Small comes with HI-Small_Patterns.txt, which names the laundering pattern behind each chain. It is parked in later.md to let explanations say "matches a fan-out pattern".

Features

Every payment is described by 25 features that use only payments from strictly before it. Amounts are converted to US dollars first (notebooks/currency.py). That way the model sees exactly what a bank would know at the moment the money lands.

Why "point-in-time" matters. If a feature counted an account's future payments, the model would learn "accounts that later send lots of money out are mules". That's true, but useless when the money has only just landed. It would score brilliantly in testing and fail for real. This data leakage is the most common mistake in fraud projects. Payments in the same minute are also excluded. The fast feature code was checked against a slow brute-force calculation on random payments, and they match exactly.

How it's fast. A helper, window_stats, sorts payments by (account, time) and uses binary search to answer questions like "how many payments did this account receive in the 24 hours before this one?" for all 4.5M payments in about a minute. Notebook: notebooks/02_features.ipynb.

The features fall into three groups. Each group answers a question a fraud analyst would ask.

The payment: is this payment itself unusual? (8)

Feature	Meaning
log_amount	Payment size in US dollars on a log scale, so huge amounts don't dominate
payment_format	ACH, cheque, card, cash, Bitcoin, wire or reinvestment
payment_currency	Currency it was paid in
cross_currency	Was it converted between currencies?
cross_bank	Did it move between two banks?
hour	Hour of day it was sent
new_pair	First ever payment between these two accounts?
pair_prior_cnt	How many times this sender has paid this receiver before

The receiver, the possible mule: is this account behaving like a mule? (10)

Feature	Meaning
r_age_h	Hours since the account was first seen; mule accounts are often new

Feature	Meaning
r_in_cnt_all	Total payments it has ever received
r_in_cnt_24h	Payments received in the last 24 hours
r_in_cnt_1h	Payments received in the last hour
r_out_cnt_all	Total payments it has ever sent
r_out_cnt_24h	Payments it sent in the last 24 hours
r_distinct_senders	How many different accounts have ever paid it
r_new_senders_24h	New senders in the last 24 hours (fan-in: money from strangers)
amt_vs_r_avg	This payment against the account's usual incoming amount
r_passthrough_24h	Share of the last 24 hours' incoming money already sent back out

The sender, the possible recruiter: is the money coming from a suspicious source? (7)

Feature	Meaning
s_age_h	Hours since the sender was first seen
s_out_cnt_all	Total payments the sender has ever made
s_out_cnt_24h	Payments the sender made in the last 24 hours
s_in_cnt_24h	Payments the sender received in the last 24 hours (money flowing through the sender too)
s_distinct_receivers	How many different accounts the sender has ever paid
s_new_receivers_24h	New accounts paid in the last 24 hours (fan-out: a recruiter spraying money to several mules)
s_passthrough_24h	Share of the sender's last 24 hours' incoming money it has already sent on (the sender may itself be a mule)

Sender fan-out is our strongest early-warning signal. If a sender is paying many new accounts, every receiver is at risk, including a first-time mule with a clean history.

Evaluation method

We train on earlier days and test once on later days, and we judge models by how much laundering they catch in the riskiest 1% of payments, not by accuracy.

Split by date, never randomly.

Split	Days	Payments	Laundering	Purpose
Warm-up	1 Sep			History only: on day 1 every account looks brand new
Train	2–6 Sep	2.09M	2,203	The model learns from these
Validation	7–8 Sep	945k	1,036	Tuning and choosing alert cut-offs
Test	9–10 Sep	845k	954	Final score, looked at once

A bank trains on last month and uses the model next month, and the split copies that. A random split would put payments from the same mule ring in both train and test. The model would then recognise specific accounts rather than learn the pattern, and its score would be inflated.

Metrics.

- **Recall at a 1% alert rate:** flag the riskiest 1% of payments and ask what share of all laundering is caught. A bank's fraud team can only review a small slice, so this is how a bank would judge us.
- **Precision at 1%:** what share of those alerts are real.
- **PR-AUC:** how well laundering is ranked above normal payments across every alert level. Random guessing scores about 0.001 here, the base rate.
- **Why not accuracy:** a model that says "never laundering" is 99.9% accurate and useless.

Models and results

XGBoost catches 69.5% of laundering in its riskiest 1% of payments, 2.7× the rules baseline. Within ACH payments alone it still catches 53.6% at a 42% hit rate, so it has learned mule behaviour, not just payment type.

Rules baseline. A hand-written checklist like one a bank might use, scoring one point per red flag (0 to 5), with payment size as a tie-breaker:

1. Paid by ACH (the format carrying 87% of laundering)
2. First ever payment between these two accounts
3. Sender has paid 3+ new accounts in the last 24 hours (fan-out)
4. Receiver has been paid by 2+ new senders in the last 24 hours (fan-in)
5. Receiving account is less than 24 hours old

XGBoost (final model). It is the standard model for tabular data, trains fast on a CPU (the VM has no GPU), handles features on very different scales, and reports how much each feature pushed each score, which our explanations need. scale_pos_weight makes each laundering example count as much as about 1,000 normal ones, rather than inventing synthetic laundering. Training stops early when validation stops improving. Notebook: notebooks/03_models.ipynb.

Isolation Forest (anomaly model). It needs no labels and flags payments that are simply unusual, which could catch laundering tricks the labels don't cover. Its score was blended with XGBoost as combined = (1 − w) × XGBoost + w × IsolationForest, with w chosen on validation days only.

Both were trained on IBM LinuxONE: XGBoost in 19.0 s (best iteration 74), Isolation Forest in 6 s.

Results on the test days (9–10 Sep, never seen in training or tuning). Random guessing would score about 1% recall at 1% alerts and 0.001 PR-AUC.

Model	Recall at 1% alerts	Precision at 1%	PR-AUC
XGBoost (final model, 25 features, USD)	**69.5%**	**7.9%**	**0.430**
Rules baseline	25.7%	2.9%	0.024
Isolation Forest only	1.1%	0.1%	0.002

Model	Recall at 1% alerts	Precision at 1%	PR-AUC
XGBoost + Isolation Forest	same as XGBoost (best blend weight on validation was 0)		

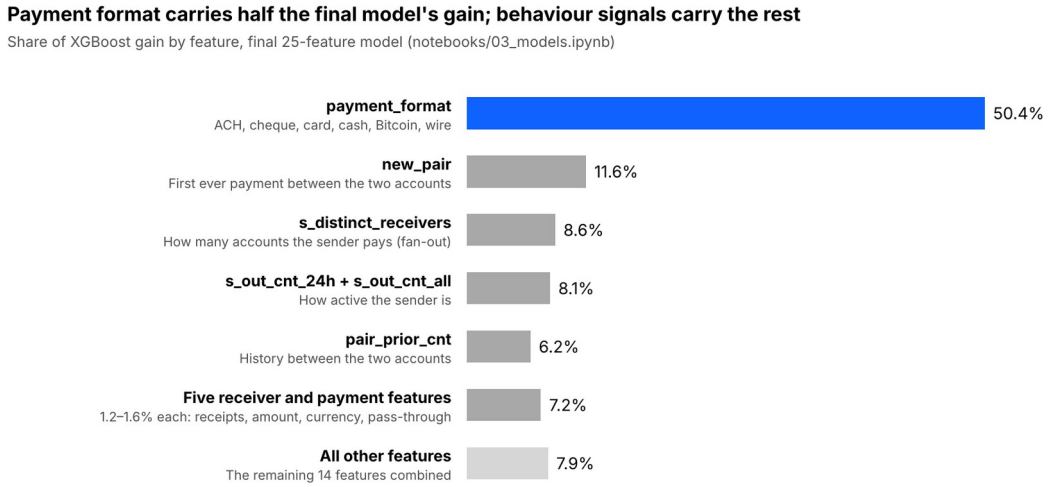
Earlier 24-feature model (amounts not converted): XGBoost 59.3% / 6.7% / 0.303; rules 25.5% / 2.9% / 0.022. The Isolation Forest rows come from that earlier run.

“Isn’t it just detecting ACH?” No. We re-scored ACH payments only (106,123 payments, 832 laundering). Here “is it ACH?” tells you nothing, and the rules collapse to near-random:

Model (ACH only)	Recall at 1%	Precision at 1%	PR-AUC
XGBoost	**53.6%**	**42.0%**	**0.492**
Rules baseline	2.0%	1.6%	0.027

Earlier model: XGBoost 38.5% / 30.2% / 0.345; rules 2.3% / 1.8% / 0.026.

What the model relies on.



Final model feature importance

XGBoost feature importance (gain), final model. Payment format fell from 58.7% of gain in the first version to 50.4%, while first-time pairs (11.6%) and sender fan-out (8.6%) rose, so the model leans more on behaviour.

Payment format dominates overall, but the ACH-only check above shows the behavioural features, such as first-time pairs and sender fan-out, do the work once format is held fixed.

Decisions from these results.

- **Isolation Forest is dropped from the score (weight 0).** On its own it was no better than random, and every blend made validation worse: recall fell from 77.7% to 63.8% at weight 0.05 and to 18.5% at 0.3. The finding is useful in itself: mule payments are ordinary-sized ACH transfers, not statistical outliers. Mules hide in plain sight, so you need a model that learns the specific laundering pattern. The final system is therefore one predictive model (XGBoost) whose reasons are written up in plain English; the generative step from the second lab is supported in the code but was not running in the final build. We report the Isolation Forest result openly.

- **We quote the test number, not validation.** Validation recall was 77.9% and test 69.5%. Patterns drift over time, which is why the test set stays untouched.

- **Precision at 1% is about 1 in 13**, as in all real fraud systems. That shaped the product: the student is contacted only at a much stricter cut-off (see Alert levels).

What changed in the final model (US-dollar pipeline)

The numbers above come from the final model. It differs from the first version in four ways:

- **Amounts in US dollars.** Every amount is converted with the rate table derived from the data (notebooks/currency.py; IBM's generator uses one fixed rate per currency, checked on all 5.08M rows), so a payment of 1,000 yen no longer looks the same as one of 1,000 pounds.

- **A 25th feature.** s_passthrough_24h measures the sender's own pass-through over the last 24 hours, matching the receiver feature r_passthrough_24h.

- **Slimmer data.** notebooks/01_cleaning.ipynb rebuilds the cleaned set in 0.48 GB instead of about 1.5 GB, after kernel crashes on the shared VM. It keeps the same 4,486,418 payments, 4,514 of them laundering.

- **Isolation Forest code removed.** Its blend weight was 0 and the live service never loaded it. Its result stays in the table above as the record of the experiment.

Effect on the untouched test days: recall at 1% rose from 59.3% to 69.5% and PR-AUC from 0.303 to 0.430. The gap between validation and test shrank (77.7%→59.3% before, 77.9%→69.5% now), which suggests the model generalises better to later days.

Alert levels

Each cut-off is the one that maximises an F-beta score on the validation days, then measured once on the test days. 7 in 10 student warnings are real mule payments, and the analyst queue is about 436 alerts a day, half of them real.

F-beta balances recall (R) and precision (P), counting recall β^2 times as much as precision:

$$F\text{-beta} = (1 + \beta^2) \times (P \times R) / (\beta^2 \times P + R)$$

- **Analyst alert: F2** (recall-weighted). Missing a mule is worse than an analyst checking an extra payment. Guard-rail: never more than 5% of payments.

- **Student warning: F0.5** (precision-weighted). Scaring an innocent student is worse than missing a case, because the analyst level still sees it.

- 2 and 0.5 are the standard values. A cost-based β would need real bank cost figures we don't have, so a more precise-looking β would just be a guess.

Results on the test days (about 845k payments, 0.1% laundering). Implemented in service/export_for_service.py.

Level	Test payments flagged	Alerts per day	Laundering caught	Alerts that are real
Analyst alert (F2)	0.10%	~436 (~220 false)	**45.2%**	**49.5%** (1 in 2)
Student warning (F0.5)	0.06%	~259 (~78 false)	**37.8%**	**69.7%** (7 in 10)
For comparison: top	2.48%	~10,500	74.7%	3.4% (1 in 30)

Level	Test payments flagged	Alerts per day	Laundering caught	Alerts that are real
1% of validation				
For comparison: top 0.1% of validation	0.09%	~360	43.1%	56.8%

Earlier 24-feature model: analyst 0.31% flagged, ~1,300 a day (~1,090 false), 45.9% caught, 16.8% real; student 0.03%, ~125 a day (~37 false), 18.6% caught, 70.8% real.

What changed and why. The first version used round cut-offs (top 1% and top 0.1% of validation scores). Moving to F-beta made both levels stricter. Total analyst recall fell from 75% to 45%, but alerts are about 15× more likely to be real (49.5% vs 3.4%) and the analyst queue is about 24× smaller. The cut-offs are also stable between validation and test (analyst 0.13% → 0.10% flagged, student 0.07% → 0.06%), unlike the old top-1% level (1% → 2.48%). With the final model, student warnings catch twice as much laundering as before (18.6% → 37.8%) at the same 7-in-10 precision; the cost is about 78 wrong warnings a day instead of 37.

The bank controls the trade-off. The threshold sweep chart (threshold_sweep.png) shows recall and precision at every possible cut-off, so a bank with more analysts can choose a looser one.

Explanations

Every alert explains itself using the model's own reasoning. Any generative step may only reword those facts, so it cannot invent a reason.

1. **Top reasons from the model.** For each scored payment, XGBoost reports how much each feature pushed the score up (SHAP contributions, pred_contribs). The service takes the strongest positive contributions in order.

2. **Plain-English wording.** Each feature has one sentence, e.g. "sender paid 7 new accounts in the last 24 hours", "first ever payment from this sender to this account", "85% of money received in the last 24 hours has already been sent on". A fact that says nothing useful to a person (such as "0 payments in the last hour") is skipped so the next reason shows instead. ACH appears on almost every alert, so it is shown as a field, not a reason. Up to three reasons are kept.

3. **Rarity, not a percentage.** The score is stated as "Only about 1 in N payments scores this high", measured against all validation payments.

4. **Two messages.** The **student message** is calm and non-accusing: it says the payment matches money-mule scam patterns, not to send it on, and to call the bank using the number on their card. The **analyst note** gives the rarity, the reasons, and a suggested action: hold outbound transfers from the receiving account, and check the sender for payments to other new accounts.

In the final build both messages come from templates filled with these reasons. The service can also pass the same reasons to IBM Granite on the VM, only when an explanation is opened so scoring never waits, but no Granite server ran, so the demo labels each note "from the model's reasons".

The web app and demos

The StudentShield web app has three pages. Every payment, score, alert level and reason shown in the two demos is real model output on the unseen test days. Only the "Student" persona is illustrative, because the dataset has no ages.

Page	Job	What it shows
/ landing page	The pitch	Hero with the warning screen and

Page	Job	What it shows
		an animated detection pipeline that ticks off each finished step; sourced problem statistics; four-step “how it works” (step 02 names the point-in-time signals); usual approach vs StudentShield; *Run live demo*
/demo story view	The experience, one real case step by step	Three panels: live payment stream, the student’s phone, the bank analyst console
/live full replay	That it works at scale on unseen data	All replayed test-day payments in time order at 5–100 per second, with live alerts, catch rate, false alarms and latency (about 1 ms per payment)

How /demo picks its case. The scoring service’s /cases endpoint searches the test days for the fan-in then send-on pattern: accounts that received at least two laundering payments and later sent money out. It ranks them by how many incoming payments reached the student level, and the demo plays the strongest. Up to 8 inbound payments and the first outbound transfer play out one every 1.6 seconds. Each one is sent to the model as it “arrives”, alongside 300 real normal payments from the same hours as background traffic.

The three panels.

- **Live payment stream:** a day-divided feed and account tiles that go green, amber (analyst level) or red (student level) from the model’s actual output.
- **Student’s phone:** follows the phone rules above. It shows the warning at the student level, flashes updated figures with a toast when new money arrives, re-asks after flagged payments, and confirms a pause.
- **Analyst console:** risk as “1 in N”, key figures, a money-flow graph (senders → account → destination, with the held transfer), a timeline with dates, the explanation (labelled “from the model’s reasons”), and every answer the student gave.

Lead time. When the outbound transfer arrives, the demo computes how long before it each warning fired. In the case played, the student was warned about 19 hours before the money moved out.

Amounts carry no £ sign because the dataset mixes currencies. The top bar shows the server’s architecture, read live, and reads **s390x** on LinuxONE.

Running it on LinuxONE

The whole system runs as three Docker containers on one LinuxONE Community Cloud VM. All of them are built on the LinuxONE s390x Jupyter image, which already contains s390x builds of NumPy and XGBoost, so nothing heavy has to compile.

Item	Detail
VM	LinuxONE Community Cloud, AI_LAB flavour: 3 vCPU, 8 GB RAM, 50 GB disk, Ubuntu 24.04 (Datathon image), s390x
notebook container	jupyterlab-image-s390x, port 38888: data loading, features, training (notebooks 01–03)
mule-service container	Flask + XGBoost scoring service on the private

Item	Detail
	Docker network mule-net, no public port; reads the model and thresholds from ~/shared/service_data
studentshield container	Flask web app, the only public port (5000); talks to mule-service over mule-net
Shared folder	~/shared on the VM, mounted into Jupyter, so the trained model passes from notebook to service and survives container rebuilds

Deployment (VM terminal, from the repository folder):

```
# once, in the Jupyter terminal: export model, thresholds and replay data
python service/export_for_service.py

# on the VM
cd ~/shared/manchester-imbz-datathon-2026 && git pull
docker network create mule-net 2>/dev/null || true

docker build -t mule-service service/
docker run -d --name mule-service --network mule-net --restart unless-stopped \
  -v ~/shared/service_data:/data mule-service

docker build -t studentshield frontend/
docker run -d --name studentshield --network mule-net --restart unless-stopped \
  -p 5000:5000 -e MODEL_URL=http://mule-service:5000 studentshield

sudo iptables -I INPUT -p tcp --dport 5000 -j ACCEPT
```

Scoring service API (service/app.py):

Endpoint	Returns
GET /health	Status, machine type (s390x), thresholds, LLM on or templates
POST /score	Score one payment: probability, level, top reasons, messages, latency
GET /alerts?limit=50	Latest alerts, newest first
GET /explain/<id>	Student warning and analyst note (Granite if configured, else template)
GET /stats	Payments scored, alerts, student warnings, catch rate, average latency
GET /cases, GET /case/<account>	Real fan-in then send-on accounts from the test days, and their payments
POST /demo/start, /demo/stop	Replay the test days inside the service

How the platform is used.

- **On-platform AI:** training and inference both run on s390x. Transaction data never leaves the platform it would live on in a bank.
- **Compute:** XGBoost trains on 2.09M payments in 19.0 s and scores in about 1 ms, on 3 vCPUs with no GPU.
- **Why IBM Z for this problem:** most of the world's card and bank transactions already run on IBM Z, so scoring them in place avoids copying sensitive data elsewhere. The Telum processor's on-chip AI accelerator is designed for inference inside the transaction, which is exactly the window StudentShield needs.

Containerisation, security and scalability

The prototype runs on one VM, but each part can be copied or moved without code changes. This is what we built, and what scaling to a bank's payment volume would still need.

- **One base image, three containers.** All three start from IBM's s390x Jupyter image, so its layers are already on the VM and NumPy and XGBoost never have to compile. The same images run on any LinuxONE machine.

- **They restart themselves.** mule-service and studentshield run with --restart unless-stopped. When the VM froze and was restarted during the event, both came back on their own; only the Jupyter container, started without a restart policy, had to be started by hand.

- **Only one door is open.** The model service has no public port and is reachable only on the private Docker network mule-net. The web app on port 5000 is the only public part of the product (Jupyter, used for development, is also open on port 38888 behind a token). In a bank, the scoring service would sit inside the payment network and never face the internet.

- **The model is data, not code.** The trained model, alert thresholds and replay file live in a shared folder mounted into mule-service. A retrained model is exported and the service restarted, with no image rebuild. Mounting the folder read-only would stop the service ever changing it.

- **Scoring is stateless, so it scales out.** Each /score call needs only that payment's features and the loaded model, and takes about 1 ms. One process can therefore score in the order of 1,000 payments a second, and more mule-service replicas can sit behind the same network name to add capacity.

- **What doesn't scale yet.** Recent alerts and running totals are held in each container's memory (the last 1,000 alerts), and the demo runs one shared story for every viewer, so replicas would need a shared store such as a database. Features are computed in batch in the notebooks; live use would need per-account rolling counters updated as each payment arrives.

- **Memory was the real limit.** On the shared 8 GB VM, loading the full CSV (about 1.5 GB in memory) crashed notebook kernels. The 00:55 cleaning notebook builds a slim dataset of 0.48 GB with compact account IDs and categories.

- **Security still to harden.** The demo runs over plain HTTP, Jupyter is reachable on port 38888 behind a token, and both app containers run as root (USER root in their Dockerfiles). A production setup would add HTTPS, run the containers as an unprivileged user and close the Jupyter port.

- **Room to grow on IBM Z.** The same containers can be given more vCPUs and memory on LinuxONE, and inference could move onto the Telum processor's on-chip AI accelerator for in-transaction scoring at bank volumes. We did not test the accelerator in this build.

Responsible design

Warning a customer about suspicious money needs care. Every design choice errs towards protecting an innocent student.

- **Tipping off.** UK law restricts telling someone they are suspected of money laundering (the "tipping off" offence under the Proceeds of Crime Act 2002). The student never sees a score, a reason, an investigation status or the dataset's answer key. The prompt is safeguarding friction of the kind banks already use before risky payments ("Has someone asked you to move this

money?") and points to help. A real deployment would need the bank's legal and financial-crime teams to approve the wording.

- **False alarms are sized and controlled.** At the analyst level about half the alerts are innocent (around 220 false alarms a day on the test days), so students are not contacted at that level. Students are contacted only at the student level, where 7 in 10 are real. The cut-off is chosen to weight a mistake against the person who bears it (F0.5), and the bank can move it along the published trade-off curve.

- **No hold without confidence.** Outgoing transfers are held only once the account reaches the student level. Pressing a button records the student's answer but does not itself trigger a hold.

- **The student is heard.** Every answer goes to the analyst, and the student is re-asked only when new flagged money arrives, at most three times.

- **Explainable by construction.** Reasons come from the model's own feature contributions. The LLM may only reword them, so it cannot invent a reason.

- **Honest evaluation.** A time-based split, point-in-time features checked against brute force, a test set used once, the validation-vs-test drop reported, the synthetic-data tail excluded, and the failed Isolation Forest result reported rather than hidden.

- **Privacy.** Payments are scored on the platform where they live, and any language model would run on the same VM. Nothing is sent to an outside service.

Ethical considerations

Responsible design covers how the product treats a flagged student. This section covers the wider questions a bank, a regulator or a students' union would ask before using it, and where our prototype falls short.

- **Fairness: behaviour can stand in for identity.** The model never sees age, nationality, ethnicity or any other protected characteristic; it sees only how money moves. But two of its features, payment_currency and cross_currency, could flag students who receive money from family abroad more often than their classmates, and international students already face more account checks. Before any real use, alert rates should be compared across groups such as home and international students, and a currency feature that raises scores without improving precision should be removed. Until then, the explanation "paid in a currency linked to higher laundering rates" should not be shown to an analyst on its own.

- **Students are protected, not profiled.** The product focuses on students because recruiters target them, not because students are suspects. The model has no age or student flag: "student" in the demo is a persona. A real bank would score every customer the same way and could use age only to choose the support it offers.

- **A person makes every serious decision.** The model never closes an account, files a report or freezes funds. At the student level it holds an outgoing transfer and puts the case in front of a human analyst. UK GDPR requires safeguards, including human review and a way to contest, when a decision based only on automated processing significantly affects someone; the analyst review and the "This is legitimate" answer are those routes. A real deployment would need the bank to confirm its lawful basis and complete a data protection impact assessment.

- **Criminals should not learn the rules.** The phone never shows scores, reasons or cut-offs, so a recruiter coaching a student cannot learn which behaviour to avoid. Reasons appear only in

the analyst view.

- **No real person's data was used.** The model was trained and tested on IBM's synthetic dataset, so the prototype raises no consent or anonymisation questions. A real version would process customers' transactions and should keep to data minimisation: the 25 features are counts, amounts and timings, not message content or location.

- **We are honest about the AI.** Results on synthetic data are labelled as synthetic. Since 22:32 the landing page makes no Granite claim the demo can't back, and the AI assistance used to write code and documentation is declared in References.

- **Low compute cost.** XGBoost trains in 19.0 s on 3 CPUs with no GPU, and no language model runs in the scoring path.

Decisions that changed during the build

Many times in the day, what we built first turned out to be wrong or weaker than it could be, and we changed it.

Each change is in the repository's commit history: 70 commits, all made during the event from 11:26 on 17 October. Most came from looking honestly at a result or at the screen from a student's point of view.

What we did first	What we changed it to	Why
The live stream kept running while a question was open on the student's phone	The stream pauses until the student answers, with a paused banner on screen	New payments arriving mid-question buried the moment that matters. Pausing keeps the question in focus while the student decides
Landing page said IBM Granite writes the analyst's case note, listed Granite among the tools, and quoted about 2 ms per payment	Wording that is true either way ("a case note built from the model's reasons"), tech list removed, about 1 ms	The demo falls back to templates whenever Granite isn't served, so the page must not promise Granite. Judges trust claims they can see; a claim the demo can't back costs more than it earns
Held the student's outgoing transfer on **any** flag, analyst level included	Hold only once the account reaches the **student** level	At the analyst level about half the alerts are innocent (49.5% precision; 5 in 6 with the earlier model). Holding there would block many innocent students; at the student level 7 in 10 are real
Re-asked the student every time any money arrived, with no limit	Re-ask only when **flagged** money arrives; stop after "Someone asked me to move it" or 3 answers	Ordinary payments shouldn't nag the student. Once they say someone asked them, a specialist takes over, and asking more adds nothing
Kept only the student's latest answer; "This is legitimate" said the money was "kept on hold for a quick check"	Keep **every** answer, with how many payments it covered and whether it changed; plain answer labels	A changed answer is itself evidence for the analyst. And the button shouldn't promise an action it doesn't take: the hold comes from the model's level, not the button
Considered letting the student mark each payment, or each sender, as "I know this person"	One question about the whole situation, asked again when new flagged money arrives (per-payment ticks parked in later.md)	The student can't refuse incoming money anyway (only the bank can recall it), and the danger is the recruiter, not any single sender. One question keeps the moment simple

What we did first	What we changed it to	Why
Showed risk as “top X% of payments”	“Only about 1 in N payments scores this high”	“Top X%” was floored at 0.01%, so it understated the rarest scores. A count of payments is also easier to grasp than a tiny percentage
Front end built against a simulated stream: a placeholder scoring formula and a scripted £2,400 case	Wired to the real model: plays the strongest real fan-in-then-send-on account from the unseen test days, every payment scored live; amounts shown without £	The front end and model were built in parallel, so the UI started on fake data. For the demo, every score on screen had to be real model output on unseen data. The dataset mixes currencies, so a £ sign would be wrong
mule-service published on public port 5000	Model service on a private Docker network (mule-net); only the web app is public	The model API doesn’t need to be on the internet. One public entry point means less to attack, which fits the “AI secured” side of the platform story
ACH listed as a reason on alerts	ACH shown as its own field, not a reason	ACH is on nearly every alert, so as a reason it told the analyst nothing and pushed out reasons that did
A sentence for every fact, including “quiet in the last hour”, “sent within the same bank”, “sent at 14:00”	Facts that say nothing useful to a person are hidden, and the next strongest reason is shown instead	Correct but meaningless reasons waste the analyst’s three lines. Phase 3 had already flagged wording like “0 earlier payments between these two accounts”

Roads not taken

At each fork we picked the option a real bank could trust over the one that would make our numbers look better. These are the choices, what we chose them over, and why.

Data and evaluation

Decision	We chose	Instead of	Why
Yardstick	A rules baseline built first	Going straight to machine learning	“70% recall” means nothing alone. Beating a bank-style rule 2.7× proves the model adds value, and if a rule had matched it we’d have known early
Threshold drift on test	Keep the cut-offs set on validation	Re-tune them on the test days	Re-tuning on test is cheating. A real bank would recalibrate on fresh data, and we say so

Model

Decision	We chose	Instead of	Why
Main model	XGBoost	Heavier models that want a GPU	It’s the standard for tabular data, trains in 19.0 s on CPUs (the VM has no GPU), copes with counts and amounts on very different scales, and reports each feature’s contribution, which our

Decision	We chose	Instead of	Why
			explanations depend on
Rare positives	Class weighting (scale_pos_weight ≈ 1,000)	Generating synthetic laundering examples	Fake examples could teach the model patterns that don't exist. Weighting keeps the data real
Proving it isn't "just ACH"	Re-score on ACH payments only	Dropping payment_format from the model	Format is a real signal and banks have it. Testing within ACH shows the behaviour features work (53.6% recall vs 2.0% for the rules) without throwing the signal away
Choosing β	Standard F2 and F0.5	A β derived from costs	A cost-based β needs real figures for an analyst's time and a wrongly warned student, which we don't have. A precise-looking β would just be a guess
Timebox	Stop tuning when the model clearly beats the baseline, or at 21:00	Chasing a few more points of recall	A working live demo on LinuxONE is worth more than 2% of recall (SCOPE.md stop rule)

Explanations and the student experience

Decision	We chose	Instead of	Why
How the student is approached	A question: "Has someone asked you to move this money?"	A notice that the account is flagged or frozen	It doesn't accuse, doesn't reveal a suspicion (tipping off), and matches scam checks banks already use
The student's answers	One answer about all the money, logged for the analyst	Per-payment or per-sender ticks	Simpler at a stressful moment, and the analyst still sees every answer and how it changed (parked in later.md)

Platform

Decision	We chose	Instead of	Why
Front-end stack	Flask serving HTML, CSS and JavaScript, polling every 0.8 s	A Python dashboard framework such as Streamlit	Full control over the design, and no risk of a dependency that won't install on s390x. Polling once a second looks live and is simpler than a streaming connection

Limitations and next steps

The prototype proves the approach end to end on synthetic data. Real accuracy, and the right cut-offs for a real bank, would need real bank data.

Limitations

- **Synthetic data.** IBM's AML set is generated, and its last week had to be dropped. Real-world precision and recall are unknown. The fixed currency rates come from the generator, so real exchange-rate movement is not modelled.

- **Payment format carries much of the signal.** It accounts for 50.4% of the final model's gain (58.7% in the first version). The ACH-only check shows the behavioural features do real work, but a bank with different payment mixes would need retraining.

- **Persona, not people.** The dataset has no ages or student status, so "student" in the demo is a label for the story.

- **Higher laundering rate than real life.** HI-Small has about 1 laundering payment in 1,000. Real banks see rates nearer IBM's LI sets (about 1 in 1,750 in LI-Large), so precision in a real bank would be lower than ours.

- **The hold is simulated.** The demo shows outgoing transfers held at the student level, but it is not connected to a payment system.

- **Recall falls over time** (77.9% on validation, 69.5% on test), so a deployed model would need regular retraining and recalibration.

With more time, we would

- Run a short survey of students on how they are approached with "easy money" offers.

- Serve IBM Granite on the VM, so the student warning and analyst note are written by a language model from the same reasons.

- Harden security: HTTPS, containers running as an unprivileged user, and the Jupyter port closed.

- Move alerts and demo state into a shared store, so several scoring containers can run side by side.

- Feed the student's answers into the analyst explanation and the Granite prompt, not just beside it.

- Name the laundering pattern in explanations (e.g. "matches a fan-out pattern") using HI-Small_Patterns.txt.

- Train on the larger HI-Medium and HI-Large datasets.

- Graph features across accounts, to flag whole mule rings rather than single links.

- Predict mule accounts at account opening, before any money arrives.

- Cross-bank detection that preserves privacy (e.g. federated learning), since mule chains cross banks.

- Move inference onto IBM Z's on-chip AI accelerator for in-transaction scoring at bank volumes.

- Test the warning wording with real students.

References and open-source used

Sources

- Fifth of suspected money mule cases involve under-21s (STV News, reporting Cifas and UK Finance data, July 2023)
- UK Finance and Cifas launch lessons (UK Finance press release with the H1 2023 Cifas figures: 23% aged 21 and under, 64% aged up to 30)
- New research reveals 65% of UK money mules are under 30 (Financial IT, January 2025; source of the Cifas 22% survey, the 37,000 accounts estimate and the NCA £10bn+ estimate)
- Altman et al., Realistic Synthetic Financial Transactions for Anti-Money Laundering Models (NeurIPS 2023 Datasets and Benchmarks), the paper behind the dataset
- UK payment fraud losses topped £1.28bn in 2025 (FinTech Futures)
- UK Finance Annual Fraud Report 2026
- jupyter-lab-ml: AI/ML on IBM LinuxONE (lab notebooks and the s390x container image)
- IBM Transactions for Anti Money Laundering (AML) dataset, HI-Small, published by IBM on Kaggle: https://www.kaggle.com/datasets/ealtman2019/ibm-transactions-for-anti-money-laundering-aml
- Project repository: ksshubhan/manchester-imbz-datathon-2026, including FINDINGS.md, SCOPE.md and the notebooks

Open-source and tools

- XGBoost: main model and SHAP contributions
- scikit-learn: metrics, and Isolation Forest (tested, then dropped)
- pandas and NumPy: data and feature engineering
- Flask: scoring service and web app
- Docker: containers on LinuxONE
- JupyterLab: model development
- IBM Granite: supported for generative explanations, but not running in the final build
- IBM Plex: typeface

AI assistance. Parts of the code and documentation were written with help from AI assistants.